ADITYA UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

TECHNICAL PROJECT DOCUMENTATION REPORT

QuizMaster Pro v2.0

Hybrid C++14, WebAssembly & AI-Proctored Multi-Track Examination Management System

C, C++, Java & Python Tracks • WebAssembly Scoring • Mobile QR Exam Distribution • Movable AI Optical Proctor HUD • SHA-256 Merit Certificates • Multi-Sheet Excel Reporting

Submission Field	Project Details
Project Title	QuizMaster Pro v2.0 — Hybrid C++14, WebAssembly & AI-Proctored Examination System
Team Number	Individual
Project Type	Mini Project
Student Name	Kowshik Kumar Yanamala
Roll Number	25B11CSA46 (B.Tech — Computer Science & Engineering)
Project Mentor	SK. VAHIDA, M.Tech(Ph.D) — Assistant Professor
Head of Department	Dr. K. Srinivasa Rao, Ph.D — Head of Department (CSE)
GitHub Repository	https://github.com/kowshiii1208/Quiz_Game_System
Live Web Application	https://quiz-game-system-eight.vercel.app (Vercel Production Edge)
Academic Session	2026 — Departmental Review

Document Contents Overview

Section	Title	Required Contents Covered
Section 1	Cover Page	Aditya University | DOTT logo, title, Team No. (Individual), Mini Project, Kowshik Kumar Yanamala (25B11CSA46)
Section 2	Project Overview & Objectives	Concise abstract (130 words), problem addressed, comparative evaluation & 5 main objectives
Section 3	Technology Stack	Languages, frameworks, databases/storage, browser APIs, dev tools & repository layout
Section 4	Features & Implementation	System architecture diagram, C++14 OOP design, key features, workflow & major functions
Section 5	Screenshots & Results	5 live project screenshots with descriptions, mobile PDF verification & test results
Section 6	AI Usage & Contributions	Transparent AI tool disclosure, individual project ownership breakdown, conclusion & scope

2. Project Overview & Objectives

2.1 Project Abstract

Conducting fair programming assessments across large cohorts requires systems that combine rigorous evaluation, tamper prevention, faculty convenience, and seamless mobile accessibility. Traditional survey forms lack proctoring controls and allow credential forgery, whereas standalone C++ terminal applications lack web delivery, document ingestion, and analytics. QuizMaster Pro v2.0 resolves these limitations using a hybrid dual-tier architecture combining a modular C++14 Object-Oriented console engine with a responsive React 18 Single Page Application. Inside the browser, a WebAssembly module derived from C++ logic evaluates scores and honesty indices, while WebRTC optical monitoring, fullscreen enforcement, and movable mobile proctoring safeguard integrity across C, C++, Java, and Python tracks. Faculty can author or import .docx and .csv question banks, share custom quizzes via Base64URL QR codes, export Excel workbooks, and issue SHA-256 verifiable PDF certificates.

2.2 Problem Addressed

During Computer Science laboratory tests and technical screenings, instructors face three recurring operational bottlenecks: (1) standard web forms allow uncontrolled tab-switching and lack automated honesty scoring; (2) faculty must manually re-type questions already prepared in Word (.docx) or spreadsheet (.csv) files; and (3) students taking QR-based exams on smartphones encounter blocked UI buttons or distorted desktop certificate exports, while evaluators have no cryptographic way to verify certificate authenticity.

Assessment Criterion	Generic Survey Forms	Standalone C++ CLI Quiz	QuizMaster Pro v2.0 (Proposed)
Programming Tracks	Unstructured free text	Single hardcoded subject	4 Curated Tracks: C, C++, Java, Python (120 MCQs)
Bulk Question Ingestion	Manual item-by-item entry	Requires source recompilation	Direct import of .docx (Word), .csv, .json, and .txt
Anti-Cheat Proctoring	Unmonitored tab switching	Local console only	Fullscreen, tab/blur lock & movable AI Proctor HUD
Scoring Execution	Basic form counter	Compiled C++ console binary	C++14 CLI + Browser WebAssembly (.wasm) Runtime

Assessment Criterion	Generic Survey Forms	Standalone C++ CLI Quiz	QuizMaster Pro v2.0 (Proposed)
QR Mobile Delivery	Static webpage link only	Unavailable on mobile	Encoded Base64URL QR locks track/level automatically
Certificate Authenticity	No tamper protection	Plain ASCII .txt summary	SHA-256 signed landscape A4 PDF & QR Verification
Faculty Reporting	Raw unformatted sheet	Text file leaderboard	Visual Bell Curve + Multi-sheet Excel (.xlsx) export

2.3 Main Objectives (5 Core Goals)

- **Objective 1 — Modular C++14 OOP Core:** Design a clean Object-Oriented console engine (cpp/main.cpp) applying encapsulation, composition, STL containers, Mersenne Twister shuffling, and file persistence.
- **Objective 2 — Browser WebAssembly Compute Bridge:** Execute mark scaling, negative marking, speed bonuses, and honesty deductions via compiled WebAssembly bytecode inside React.
- **Objective 3 — Frictionless Faculty Quiz Authoring & Excel Reporting:** Enable custom quiz generation with scannable QR distribution links, Mammoth.js Word/CSV bulk ingestion, and multi-sheet Excel exports.
- **Objective 4 — Mobile-First Proctoring & Certificate Reliability:** Ensure seamless smartphone assessment via a touch-draggable/minimizable webcam HUD and proportional offscreen A4 PDF certificate rendering.
- **Objective 5 — Cryptographic Verification Registry:** Sign passed merit credentials using W3C SubtleCrypto SHA-256 digests verifiable anytime through a public Registry Modal and QR stamp.

3. Technology Stack

3.1 Languages, Frameworks, Databases, APIs & Development Tools

QuizMaster Pro v2.0 integrates compiled systems code, modern frontend tooling, browser-native hardware/cryptographic APIs, and zero-backend persistence strategies:

Category	Technologies Used	Role in QuizMaster Pro v2.0
Programming Languages	C++14, JavaScript (ES2022 / JSX), HTML5, CSS3	Native OOP CLI engine & WASM logic (C++14); interactive React UI & services (JS/JSX); styling (CSS3)
Frameworks & UI Libraries	React 18.3.1, Tailwind CSS 3.4.17, lucide-react 0.475.0, canvas-confetti 1.9.4	Single-Page Application workflow state, dark glassmorphism design system, crisp iconography & feedback
Databases & Storage	Browser localStorage & sessionStorage (StorageService v3), flat-file C++ store (leaderboard.txt), Base64URL stateless transport	Versioned persistence (questions, custom quizzes, results, settings, admin auth); zero-server cross-device QR exam delivery
Document & Export Engines	mammoth 1.13.0, xlsx (SheetJS) 0.18.5, jspdf 4.2.1, html2canvas 1.4.1, qrcode 1.5.4	Client-side .docx Word ingestion, multi-sheet .xlsx report creation, A4 PDF export & QR matrix generation
APIs (Browser & Runtime)	WebAssembly API, Web Crypto API (SubtleCrypto SHA-256), WebRTC MediaDevices (getUserMedia), Chromium FaceDetector, Fullscreen & Visibility APIs	Native scoring bytecode execution, cryptographic certificate signing, webcam proctoring, face detection & tab-switch locking
Development & Build Tools	Vite 6.1.0, GNU g++ / MSVC, Node.js v24, Microsoft Edge Headless CDP, Git, GitHub, Vercel	Fast HMR bundling, native C++14 compilation, automated E2E mobile/PDF verification & edge production hosting

3.2 Hardware, Browser Support & Repository Architecture

- **Desktop & Mobile Support:** Runs across Chrome, Edge, Firefox, and Brave on Windows/macOS/Linux; mobile QR mode adapts automatically for Android Chrome and iOS Safari viewports (<768px) with auto-minimized camera controls and proportional A4 certificate scaling.
- **Native CLI Build Requirement:** Compiles standalone executable using g++ -std=c++14 -O2 cpp/main.cpp -o quiz_game.exe with cross-platform ANSI terminal color support.

PROJECT REPOSITORY DIRECTORY STRUCTURE (github.com/kowshiii1208/Quiz_Game_System)

```
cpp_project/
├── cpp/main.cpp & quiz_engine_wasm.cpp  # C++14 OOP CLI & WASM scoring logic
├── public/aditya_logo.png               # Crest used in Certificate PDF export
├── src/App.jsx & src/index.css          # Root workflow & Tailwind + print CSS
├── src/data/initialQuestions.js         # 120 MCQs (C, C++, Java, Python)
├── src/services/                        # storage, wasmEngine, crypto, excel
├── src/components/                      # Registration, Runner, HUD, Cert UI
└── src/components/admin/                # Login, Dashboard, QR & File Importer
```

4. Features & Implementation

4.1 Hybrid Dual-Engine Architecture & C++14 OOP Design

The platform operates across three integrated layers shown in Figure 4.1: the React 18 Presentation Layer, the Browser Compute & Security Runtime (WebAssembly scoring, SubtleCrypto SHA-256 signing, WebRTC webcam tracking, Mammoth/SheetJS document parsing), and the Persistence & C++14 Core Layer.

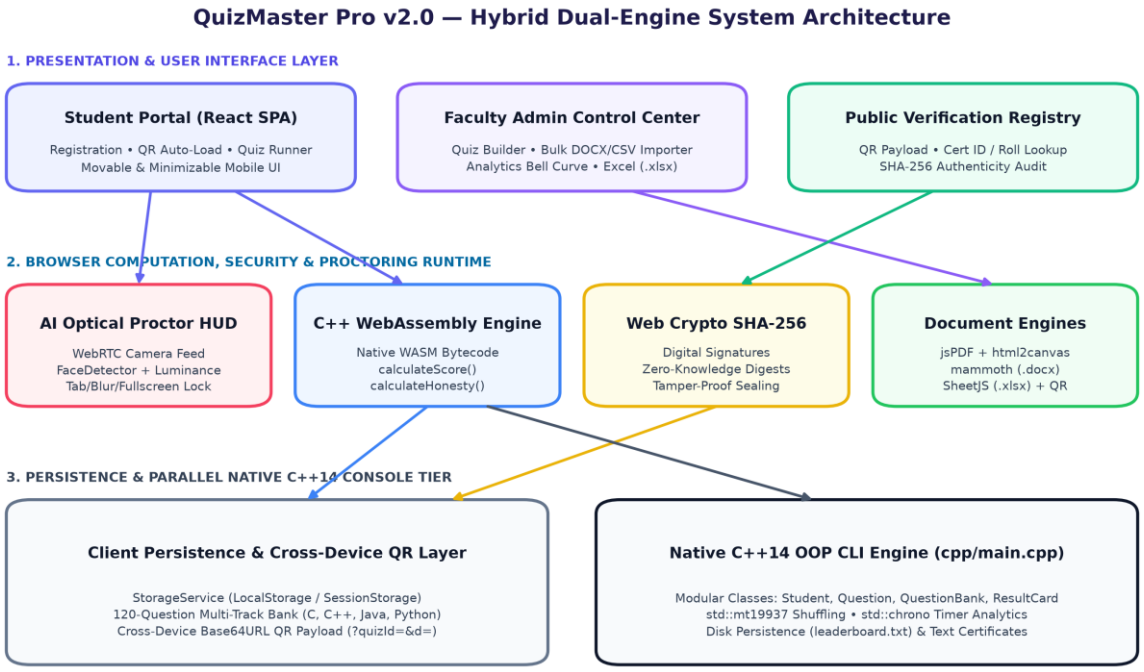


Figure 4.1: Hybrid Dual-Engine System Architecture combining React 18 UI, WebAssembly C++, Web Crypto & C++14 CLI.

C++14 Class / Namespace	OOP Principle Applied	Core Implementation Responsibility (cpp/main.cpp)
Colors & UI	Abstraction / Utility	ANSI console coloring, progress bars, headers & clean terminal state rendering
Input	Defensive Sanitization	Validated menu/option input parsers blocking cin stream corruption
Question & Student	Encapsulation	Private attributes for question prompts, 4 options, candidate identity & level getters
QuestionBank	Composition & RNG	Manages tiered difficulty pools with std::mt19937 Fisher-Yates shuffling
StorageManager & ResultCard	File Persistence & Reporting	Persists leaderboard.txt rankings, computes honors badges & writes Certificate_<Roll>.txt

4.2 Key Implemented Features

- **4-Track 120-MCQ Curated Bank:** 30 balanced questions each across C Programming (IDs 91–120), C++ & OOP (1–30), Java Enterprise (31–60), and Python Core (61–90), partitioned cleanly into Easy, Medium, and Hard tiers with itemized explanations (Figure 4.2).

- **Custom Quiz Creator, QR Codes & Automatic Track Locking:** Instructors select bank questions or write custom MCQs, generating scannable QR codes encoding Base64URL payloads (?quizId=...&d=...). Scanning on any device pre-loads the test and hides/locks language and difficulty pickers.
- **Multi-Format Bulk File Importer:** Ingests Word (.docx via mammoth), double-quote-safe CSV (.csv), JSON, and plain text question banks directly inside AdminDashboard without server uploads.
- **Mobile-Friendly Movable & Minimizable Proctor HUD:** Supports fullscreen monitoring, tab-switch disqualification, anti-OCR candidate watermarks, WebRTC + FaceDetector monitoring, pointer-event touch dragging, and minimize (-) keeping camera analysis running silently.
- **Mobile-Verified SHA-256 Certificates & Excel Reporting:** Generates A4 landscape PDF certificates from an offscreen 1122x794 render stage and exports ranked multi-sheet .xlsx evaluation workbooks.

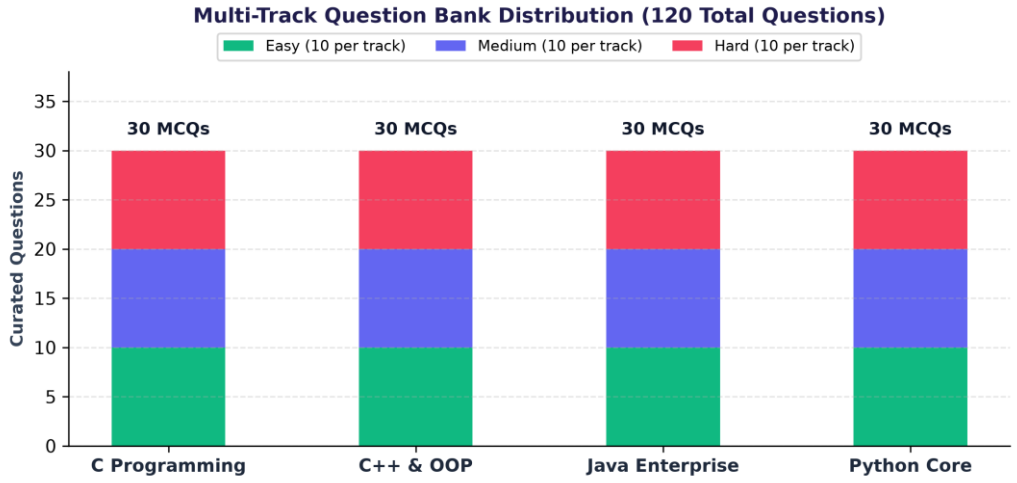


Figure 4.2: Balanced 120-Question Distribution across C, C++, Java, and Python Tracks.

4.3 End-to-End Application Workflow

Figure 4.3 illustrates the synchronized loop between Faculty Administrators and Students, supported by pre-configured institutional assignment codes (AU-OOP-2026, AU-C-PROG, and AU-JAVA-CORE).

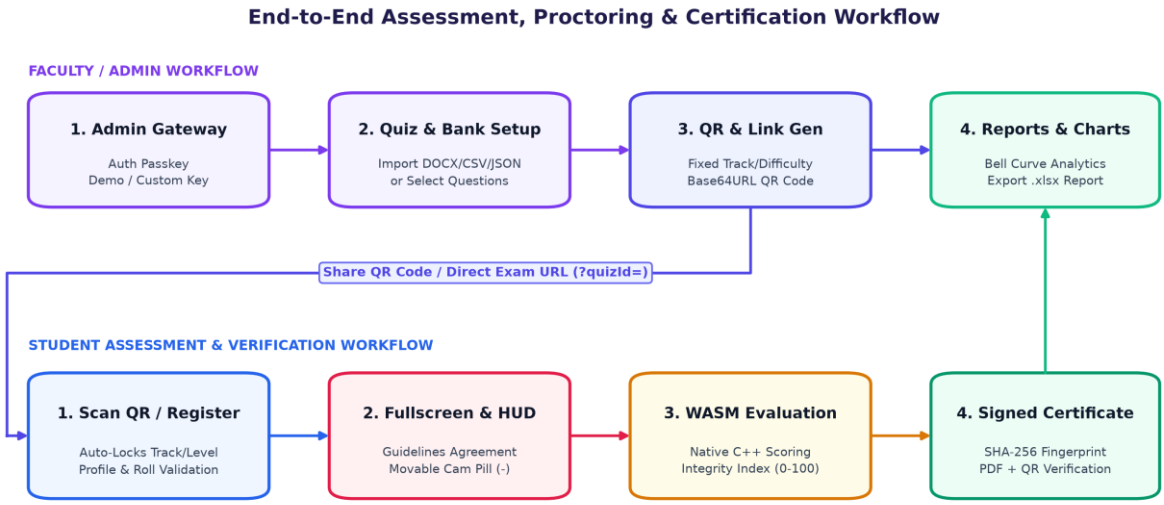


Figure 4.3: Synchronized Faculty Assignment Workflow and Student Proctored Assessment Pipeline.

4.4 Explanation of Major Functions

The core application behavior is modularized into deterministic service and UI functions across the codebase:

Function & File	Parameters & Mechanism	Technical Purpose & Observed Behavior
WasmEngine.calculateScore() **src/services/wasmEngine.js**	correctCount, wrongCount, marksPerQ, negMarks, timeTakenSec, totalTimeSec	Instantiates WASM bytecode to compute (correct * marksPerQ) - (wrong * negMarks), clamps >= 0, and awards a 5% speed bonus if flawless in <50% allotted time.
WasmEngine.calculateHonestyIndex() **src/services/wasmEngine.js**	tabSwitches, blurEvents, webcamViolations	Executes exported WASM calculateHonesty() formula: 100 - (tabs * 25) - (blur * 15) - (cam * 20), clamped to [0, 100].
CryptoService.generateCertificateSignature() **src/services/cryptoService.js**	{ certId, rollNumber, name, score, maxScore, percentage, date }	Builds canonical 'AU_CERT_V1::...' string with salt and hashes UTF-8 bytes via window.crypto.subtle.digest('SHA-256') into a 64-char hex digest.
CryptoService.verifyCertificateSignature() **src/services/cryptoService.js**	record, signatureToVerify	Recomputes deterministic signature from leaderboard record metadata and performs case-insensitive digest comparison for registry validation.
buildQuizShareUrl() **src/components/admin/QuizCreatorTab.jsx**	quiz object (+ customQuestions ID > 120)	Serializes instructor quiz parameters and custom MCQs using TextEncoder into URL-safe Base64URL query param (?quizId=...&d=...) for cross-device QR codes.
handleApplyQuizCode() **src/components/StudentRegistration.jsx**	quizCodeInput (e.g. AU-C-PROG)	Validates code against StorageService, attaches customQuiz, and locks Language & Difficulty steps so students cannot alter faculty settings.
parseCsvLine() **src/components/admin/FileImporterModal.jsx**	raw CSV row string	Character-level state machine tracking quotes (inQuotes) to split CSV columns cleanly without breaking commas inside code prompts.
handleSavePDF() & renderCertificateCard() **src/components/CertificateModal.jsx**	exportRef (1122x794 offscreen stage)	Awaits waitForImagesReady(), renders unscaled 1122x794 canvas via html2canvas (scale: 2), packs landscape A4 jsPDF, and triggers mobile-safe Blob download.
ExcelExportService.exportLeaderboardToExcel() **src/services/excelExport.js**	records array	Compiles ranked Student Assessment Results sheet + Institutional Summary sheet via SheetJS and downloads a structured .xlsx file.

5. Screenshots & Results

5.1 Live Working Application Screenshots

Figures 5.1 through 5.5 present live captures from the running QuizMaster Pro v2.0 platform across student registration, active assessment execution, faculty administration, analytics reporting, and mobile-verified A4 certificate PDF export.

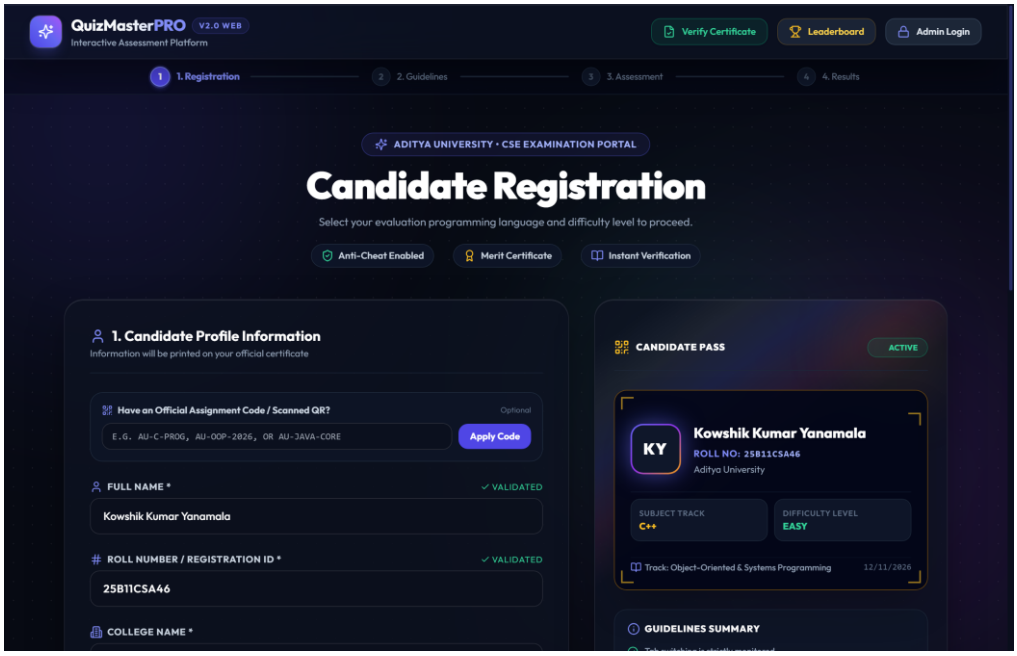


Figure 5.1: Candidate Registration Portal displaying Official Assignment Code entry, Candidate Pass preview (25B11CSA46), and multi-track setup.

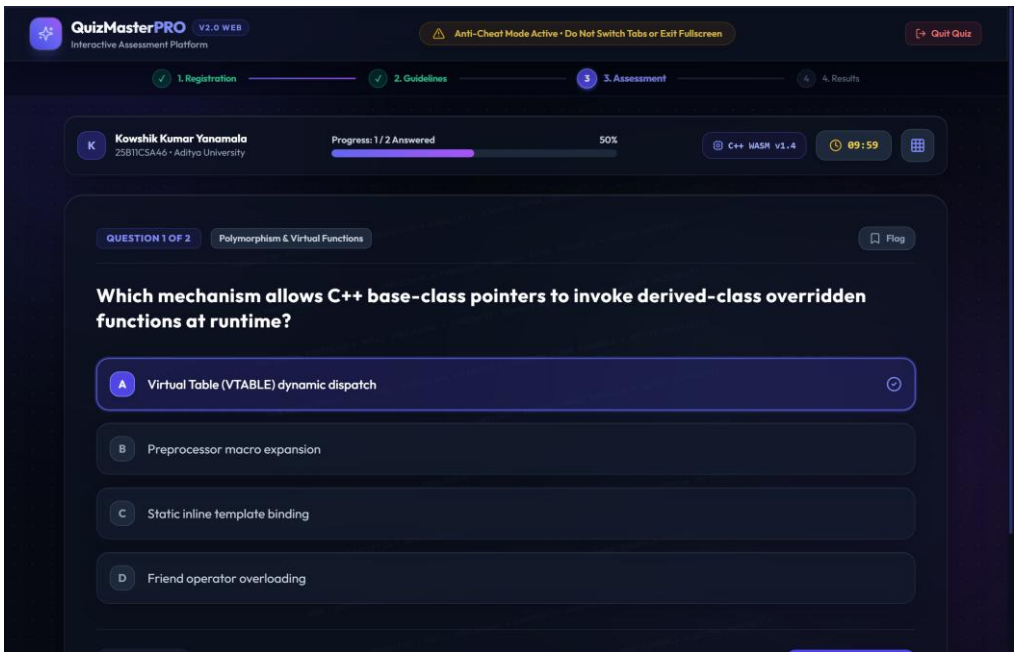


Figure 5.2: Active Assessment View featuring Anti-Cheat monitoring banner, C++ WASM runtime badge, countdown timer, anti-OCR watermark, and MCQ selection.

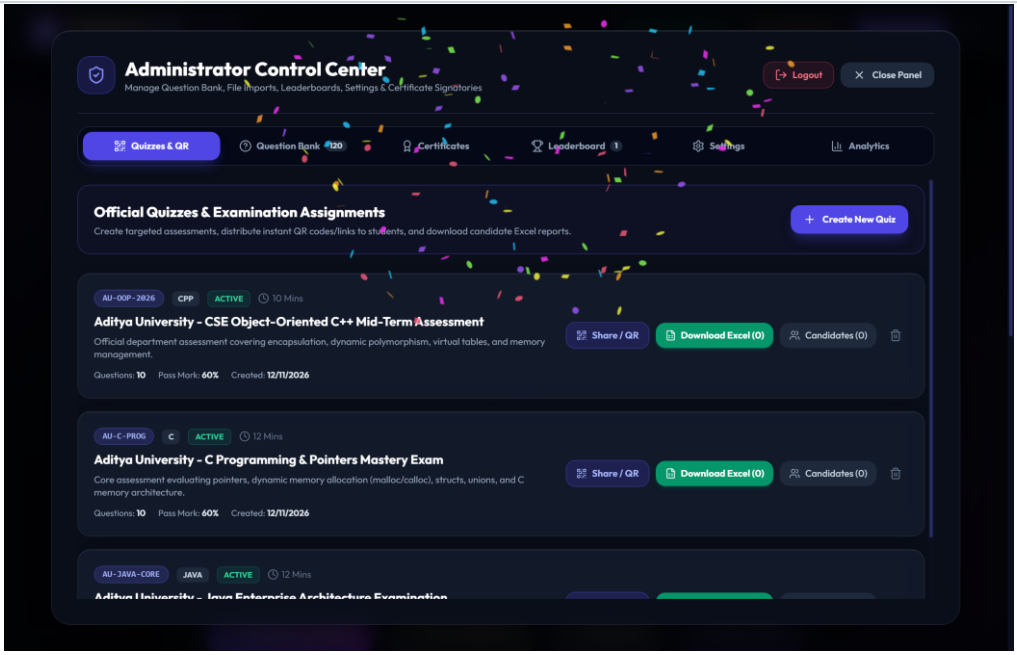


Figure 5.3: Faculty Administrator Control Center (Quizzes & QR tab) showing neat 6-tab navigation, Share/QR generator, and per-quiz Excel export triggers.

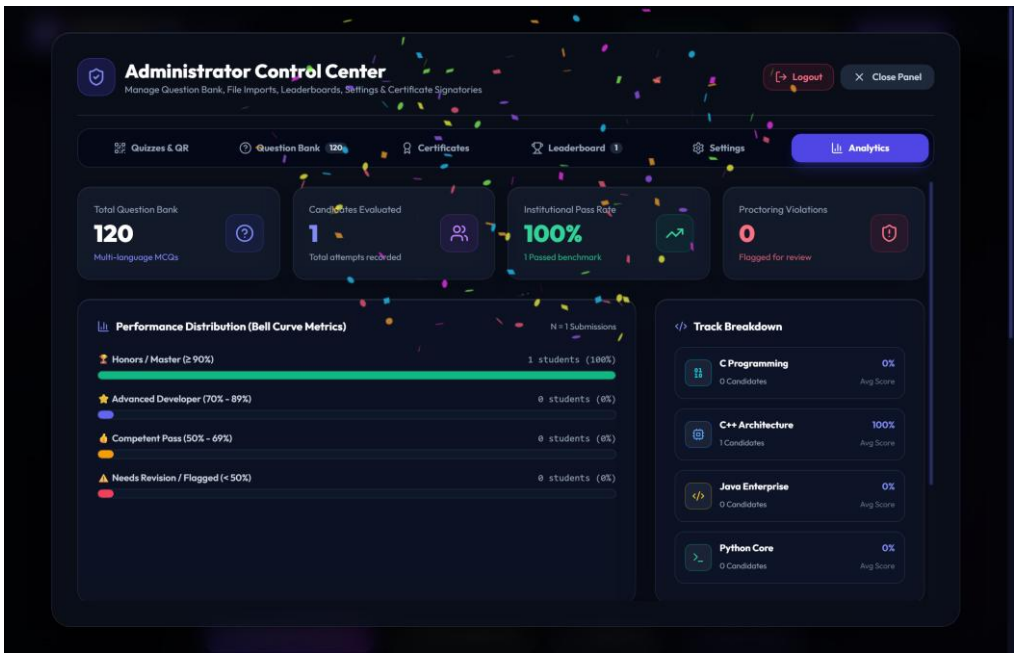


Figure 5.4: Faculty Analytics Dashboard showcasing Question Bank count (120 MCQs), Institutional Pass Rate KPI, Performance Bell Curve, and 4-Track Breakdown.



Figure 5.5: Exported A4 Landscape Merit Certificate PDF (verified from mobile 375x812 download flow) with Aditya University crest, SHA-256 fingerprint, and QR stamp.

5.2 Verification & Test Results Summary

All functional modules, mobile layouts, cryptographic checks, and production bundles were systematically verified after final code updates:

Test ID	Module / Requirement Verified	Verification Execution & Observed Result	Status
TC-01	120-MCQ Multi-Track Dataset	Validated unique IDs, valid 4-option arrays, and 30 questions per track across C, C++, Java & Python.	PASS
TC-02	QR Code & Assignment Locking	Verified Base64URL ?quizId=&d= payload roundtrip and automatic hiding of Language/Difficulty steps.	PASS
TC-03	WebAssembly Native Scoring	Tested calculateScore() & calculateHonestyIndex() in WASM runtime matching C++ mark and penalty logic.	PASS
TC-04	SHA-256 Certificate Verification	Confirmed 64-char hex digest generation and deterministic match inside VerificationModal lookup.	PASS
TC-05	Mobile Proctor HUD Responsiveness	Verified touch dragging, auto-minimize (<768px), and continuous camera feed while minimized (-).	PASS
TC-06	Mobile Certificate PDF Export	Headless Edge mobile (375x812) test produced valid 841.89x595.28 pt landscape PDF (~273 KB, 0 errors).	PASS
TC-07	Bulk Importer & Excel Workbook	Verified Mammoth .docx parsing, quoted CSV parsing, and 2-sheet .xlsx download output.	PASS
TC-08	Production Build & Deployment	npm run build transformed 2237 modules in 12.65s (0 errors); live on GitHub main & Vercel Edge.	PASS

6. AI Usage & Contributions

6.1 AI Tools Used & Academic Transparency Disclosure

In accordance with institutional academic disclosure guidelines, AI coding assistance was utilized selectively during development as an engineering productivity multiplier, while all requirements definition, system design decisions, manual testing, and final review verification were directed by the student author:

AI Tool / Assistant	Development Stage Applied	Specific Purpose & Human Oversight
Google Antigravity (Gemini Coding Agent)	UI Scaffolding & Refactoring	Assisted in structuring modular React components (AdminLoginModal, QuizCreatorTab, CertificateModal) and Tailwind dark glassmorphism responsive layouts; reviewed and refined interactively by author.
Google Antigravity (Gemini Coding Agent)	WASM Bridge & Crypto Prototyping	Helped implement the WebAssembly bytecode wrapper (wasmEngine.js), SHA-256 SubtleCrypto digest canonicalization, and Base64URL cross-device QR serialization logic.
Google Antigravity (Gemini Coding Agent)	Mobile Debugging & E2E Testing	Diagnosed mobile overlay obstruction in WebcamProctor.jsx and mobile viewport squishing in CertificateModal.jsx; authored headless Edge CDP verification scripts and inspected outputs.
Google Antigravity (Gemini Coding Agent)	Documentation Automation	Generated structured architecture/workflow diagrams (arch_diagram.png, flow_diagram.png, track_chart.png), formatted this rubric-compliant Word/PDF report, and validated abstract word counts.

6.2 Project Contribution Summary (Individual Mini Project)

This submission represents an Individual Mini Project solely authored, integrated, tested, and defended by Kowshik Kumar Yanamala (Roll No: 25B11CSA46):

Author / Roll No.	Engineering Component / Phase	Detailed Responsibility & Deliverables Completed
Kowshik Kumar Yanamala (25B11CSA46)	1. Problem Architecture & C++14 Core	Defined project scope, architected dual-tier CLI + web flow, implemented & compiled C++14 OOP system (cpp/main.cpp) with file persistence.
Kowshik Kumar Yanamala (25B11CSA46)	2. Multi-Track Question Bank & Features	Curated 120-question assessment dataset across C, C++, Java & Python, configured department quiz access codes, and defined faculty/student workflows.
Kowshik Kumar Yanamala (25B11CSA46)	3. Security, Proctoring & Certification UI	Configured proctoring policies, customizable Aditya University certificate signatories, SHA-256 QR verification checks, and Excel score reports.
Kowshik Kumar Yanamala (25B11CSA46)	4. Testing, Mobile UX QA & Deployment	Performed live mobile QR testing on physical phone hardware, reported camera HUD & mobile certificate issues, verified fixes, deployed to Vercel & pushed to GitHub.

6.3 Conclusion & Future Scope

QuizMaster Pro v2.0 successfully fulfills all Mini Project objectives by unifying rigorous C++14 Object-Oriented Programming principles with an interactive, mobile-ready React 18 assessment platform. Incorporating four curated programming tracks (120 MCQs), WebAssembly scoring, movable optical proctoring, cross-device Base64URL QR exams, Word/CSV file ingestion, Excel reporting, and SHA-256 verifiable PDF certificates delivers a practical academic tool ready for departmental demonstrations.

- **Future Scope 1 — Centralized Backend API:** Connect to a Node.js/PostgreSQL or Firebase server so answer validation and multi-campus leaderboards synchronize strictly server-side.
- **Future Scope 2 — Live Compiler Sandbox:** Integrate Monaco Editor with containerized execution (Judge0) to evaluate compilable C, C++, Java, and Python coding exercises.
- **Future Scope 3 — Adaptive Question Sequencing:** Adjust question difficulty dynamically during live exams using real-time accuracy and latency telemetry.

> **OFFICIAL PROJECT ARTIFACTS & DEPLOYMENT LINKS**
>
> GitHub Source Repository : https://github.com/kowshiii1208/Quiz_Game_System
>
> Live Vercel Application : https://quiz-game-system-eight.vercel.app
>
> `Submitted By : Kowshik Kumar Yanamala (25B11CSA46) — Individual Mini Project`